

Федеральное государственное автономное образовательное
учреждение высшего образования «Санкт-Петербургский
национальный исследовательский Университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №4
По вычислительной математике
Вариант №2

**«Аппроксимация функции методом наименьших
квадратов»**

Выполнил: Гладышев Иван Станиславович
Группа: Р3213

Принимающий работу: Бострикова Дарья Константиновна

Санкт-Петербург 2026

Содержание

1	Цель лабораторной работы	2
2	Порядок выполнения работы	3
2.1	Вычислительная часть лабораторной работы	3
2.2	Программная реализация задачи	4
3	Рабочие формулы метода наименьших квадратов	6
3.1	Общая постановка задачи МНК	6
3.2	Линейная аппроксимация: $\varphi(x) = ax + b$	6
3.3	Квадратичная аппроксимация: $\varphi(x) = a_0 + a_1x + a_2x^2$	7
3.4	Линеаризация нелинейных функций	9
3.5	Метрики качества аппроксимации	9
4	Вычислительная часть лабораторной работы	10
4.1	Исходные данные	10
4.2	Таблица значений функции	10
4.3	Линейная аппроксимация $\varphi_1(x) = ax + b$	10
4.4	Квадратичная аппроксимация $\varphi_2(x) = a_0 + a_1x + a_2x^2$	11
4.5	Сравнение и выбор наилучшей модели	11
4.6	Графики	12
5	Листинг программы	13
6	Результаты выполнения программы	19
6.1	Тест 1:	19
6.2	Тест 2:	23
6.3	Тест 3: Некорректные данные (обработка ошибок)	28
7	Выводы	29

1 Цель лабораторной работы

Цель лабораторной работы: найти функцию, являющуюся наилучшим приближением заданной табличной функции по методу наименьших квадратов. Задачи:

- сформировать таблицу значений функции $y = \frac{15x}{x^4 + 2}$ на интервале $x \in [0, 4]$ с шагом $h = 0.4$;
- построить линейное $\varphi_1(x) = ax + b$ и квадратичное $\varphi_2(x) = a_0 + a_1x + a_2x^2$ приближения по методу наименьших квадратов;
- вычислить среднеквадратичные отклонения δ для каждой аппроксимирующей функции и выбрать наилучшее приближение;
- реализовать программное исследование шести типов функций (линейная, полиномиальная 2-й и 3-й степени, экспоненциальная, логарифмическая, степенная);
- вычислить коэффициенты корреляции Пирсона r и детерминации R^2 для оценки качества аппроксимации;
- автоматизировать выбор наилучшей модели и визуализировать результаты.

2 Порядок выполнения работы

2.1 Вычислительная часть лабораторной работы

Вычислительная часть выполняется вручную (или с помощью калькулятора/скрипта) и включается непосредственно в текст отчёта.

1. Формирование таблицы табулирования. Сформировать таблицу значений заданной функции на указанном интервале с заданным шагом. Количество точек — одиннадцать.
2. Построение линейной аппроксимации. Построить линейную модель методом наименьших квадратов:
 - вычислить необходимые суммы по исходным данным;
 - составить и решить систему нормальных уравнений;
 - найти коэффициенты аппроксимирующей функции.
3. Построение квадратичной аппроксимации. Построить квадратичную модель методом наименьших квадратов:
 - вычислить дополнительные суммы по исходным данным;
 - составить и решить систему уравнений;
 - найти коэффициенты аппроксимирующей функции.
4. Вычисление метрик качества. Для каждой модели вычислить меру отклонения и среднеквадратичное отклонение. Ответы округлить до трёх знаков после запятой.
5. Выбор наилучшего приближения. Сравнить значения среднеквадратичного отклонения для линейной и квадратичной моделей; наилучшей считать модель с минимальным значением этой метрики.
6. Построение графиков. Построить на одном графике исходную функцию, линейную аппроксимацию и квадратичную аппроксимацию для визуального сравнения.
7. Подробные вычисления в отчёте. Привести все промежуточные расчёты: значения сумм, систему уравнений, её решение, значения аппроксимирующей функции в узлах, отклонения.

2.2 Программная реализация задачи

Программная часть реализуется на языке программирования по выбору и включает следующий функционал:

1. Ввод исходных данных. Предусмотреть возможность ввода таблицы значений из файла или с консоли. Объём данных — от восьми до двенадцати точек.
2. Реализация метода наименьших квадратов. Реализовать расчёт коэффициентов для шести типов аппроксимирующих функций:
 - линейная;
 - квадратичная;
 - кубическая;
 - экспоненциальная;
 - логарифмическая;
 - степенная.
3. Вывод результатов расчётов. Программа должна выводить в консоль или файл:
 - найденные коэффициенты для каждой аппроксимирующей функции;
 - меру отклонения и среднеквадратичное отклонение;
 - таблицу значений: аргумент, исходное значение, значение аппроксимации, отклонение.
4. Расчёт коэффициента корреляции Пирсона. Для линейной зависимости вычислить коэффициент корреляции для оценки силы линейной связи.
5. Расчёт коэффициента детерминации. Вычислить коэффициент детерминации и вывести сообщение о качестве аппроксимации в зависимости от полученного значения.
6. Автоматический выбор наилучшей модели. Программа должна автоматически определять и выводить наилучшую аппроксимирующую функцию по критерию минимального среднеквадратичного отклонения.
7. Визуализация результатов. Построить графики всех аппроксимирующих функций вместе с исходными точками. Оси координат должны иметь запас для наглядного отображения всего исследуемого интервала.

-
8. Тестирование программы. Протестировать программу на различных наборах данных: корректные данные, некорректные данные, данные с выбросами и шумом.

3 Рабочие формулы метода наименьших квадратов

3.1 Общая постановка задачи МНК

Пусть задана таблица точек $\{(x_i, y_i)\}_{i=1}^n$. Требуется найти параметры аппроксимирующей функции $\varphi(x) = \varphi(x, a_0, a_1, \dots, a_m)$, минимизирующие сумму квадратов отклонений:

$$S = \sum_{i=1}^n [\varphi(x_i) - y_i]^2 \rightarrow \min$$

Необходимое условие минимума: частные производные S по параметрам равны нулю:

$$\frac{\partial S}{\partial a_k} = 0, \quad k = 0, 1, \dots, m$$

3.2 Линейная аппроксимация: $\varphi(x) = ax + b$

Система нормальных уравнений:

$$\begin{cases} a \sum_{i=1}^n x_i^2 + b \sum_{i=1}^n x_i = \sum_{i=1}^n x_i y_i \\ a \sum_{i=1}^n x_i + b \cdot n = \sum_{i=1}^n y_i \end{cases}$$

Введём обозначения:

$$S_X = \sum x_i, \quad S_{XX} = \sum x_i^2, \quad S_Y = \sum y_i, \quad S_{XY} = \sum x_i y_i$$

Решение по правилу Крамера:

$$\Delta = S_{XX} \cdot n - S_X^2, \quad \Delta_1 = S_{XY} \cdot n - S_X \cdot S_Y, \quad \Delta_2 = S_{XX} \cdot S_Y - S_X \cdot S_{XY}$$

$$a = \frac{\Delta_1}{\Delta}, \quad b = \frac{\Delta_2}{\Delta}$$

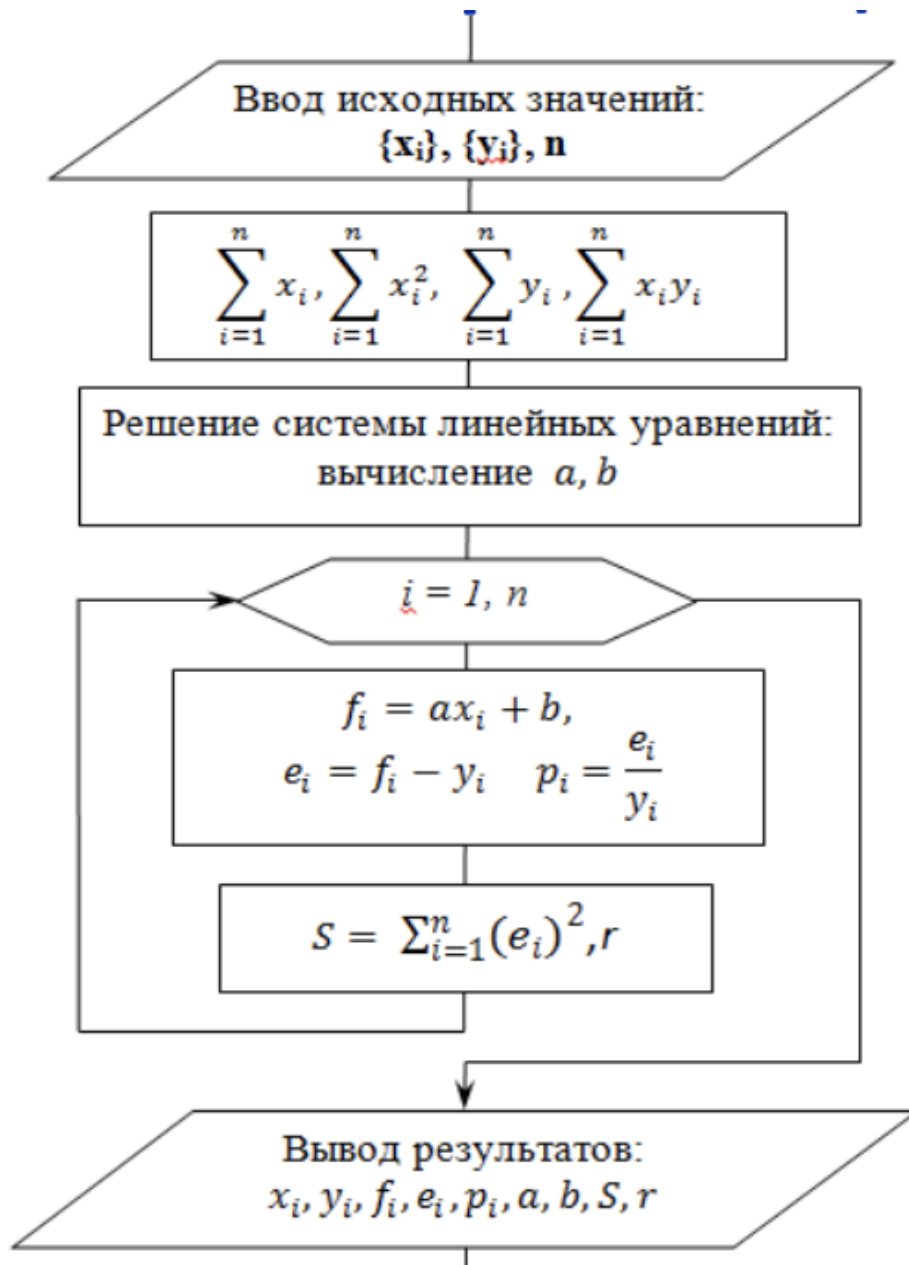


Рис. 1: Блок-схема метода наименьших квадратов для линейной аппроксимации

3.3 Квадратичная аппроксимация: $\varphi(x) = a_0 + a_1x + a_2x^2$

Система нормальных уравнений (3×3):

$$\begin{cases} a_0n + a_1S_X + a_2S_{XX} = S_Y \\ a_0S_X + a_1S_{XX} + a_2S_{XXX} = S_{XY} \\ a_0S_{XX} + a_1S_{XXX} + a_2S_{XXXX} = S_{X^2Y} \end{cases}$$

где $S_{XX} = \sum x_i^2$, $S_{XXX} = \sum x_i^3$, $S_{XXXX} = \sum x_i^4$, $S_{X^2Y} = \sum x_i^2 y_i$.



Рис. 2: Блок-схема алгоритма квадратичной аппроксимации

3.4 Линеаризация нелинейных функций

Таблица 1: Линеаризация нелинейных моделей

Вид	После линеаризации	Подстановка	Обратный переход
$y = ax^b$	$\ln y = \ln a + b \ln x$	$Y = \ln y, X = \ln x$	$a = e^A, b = B$
$y = ae^{bx}$	$\ln y = \ln a + bx$	$Y = \ln y, X = x$	$a = e^A, b = B$
$y = a \ln x + b$	$y = aX + b$	$X = \ln x$	$a = A, b = B$

3.5 Метрики качества аппроксимации

- Мера отклонения: $S = \sum_{i=1}^n (\varphi(x_i) - y_i)^2$
- Среднеквадратичное отклонение: $\delta = \sqrt{\frac{S}{n}}$ (чем меньше — тем лучше)
- Коэффициент корреляции Пирсона (для линейной):

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}, \quad -1 \leq r \leq 1$$

- Коэффициент детерминации:

$$R^2 = 1 - \frac{\sum (y_i - \varphi(x_i))^2}{\sum (y_i - \bar{y})^2}$$

Интерпретация R^2 :

- $R^2 \geq 0.95$ — высокая точность;
- $0.75 \leq R^2 < 0.95$ — удовлетворительно;
- $R^2 < 0.75$ — модель требует улучшения.

4 Вычислительная часть лабораторной работы

4.1 Исходные данные

$$y = \frac{15x}{x^4 + 2}, \quad x \in [0, 4], \quad h = 0.4, \quad n = 11 \text{ точек}$$

4.2 Таблица значений функции

Таблица 2: Таблица значений $y = \frac{15x}{x^4 + 2}$

i	x_i	y_i
1	0.0	0.000
2	0.4	2.927
3	0.8	5.556
4	1.2	6.818
5	1.6	6.250
6	2.0	5.000
7	2.4	3.750
8	2.8	2.804
9	3.2	2.128
10	3.6	1.639
11	4.0	1.282

4.3 Линейная аппроксимация $\varphi_1(x) = ax + b$

Вычисляем суммы:

$$\begin{aligned} S_X &= \sum x_i = 22.0, & S_{XX} &= \sum x_i^2 = 61.6, \\ S_Y &= \sum y_i = 38.154, & S_{XY} &= \sum x_i y_i = 71.234 \end{aligned}$$

Система уравнений:

$$\begin{cases} 61.6a + 22.0b = 71.234 \\ 22.0a + 11b = 38.154 \end{cases}$$

Решение:

$$\begin{aligned} \Delta &= 61.6 \cdot 11 - 22.0^2 = 193.6, \\ \Delta_1 &= 71.234 \cdot 11 - 22.0 \cdot 38.154 = -52.106, \\ \Delta_2 &= 61.6 \cdot 38.154 - 22.0 \cdot 71.234 = 781.026 \\ a &= \frac{-52.106}{193.6} \approx -0.269, & b &= \frac{781.026}{193.6} \approx 4.034 \end{aligned}$$

Итоговая модель:

$$\varphi_1(x) = -0.269x + 4.034$$

Мера отклонения и δ :

$$S_1 = \sum (\varphi_1(x_i) - y_i)^2 = 42.187, \quad \delta_1 = \sqrt{\frac{42.187}{11}} \approx \mathbf{1.958}$$

4.4 Квадратичная аппроксимация $\varphi_2(x) = a_0 + a_1x + a_2x^2$

Дополнительные суммы:

$$S_{XXX} = \sum x_i^3 = 193.6, \quad S_{XXXX} = \sum x_i^4 = 647.68, \quad S_{X^2Y} = \sum x_i^2 y_i = 156.789$$

Система 3×3 :

$$\begin{cases} 11a_0 + 22.0a_1 + 61.6a_2 = 38.154 \\ 22.0a_0 + 61.6a_1 + 193.6a_2 = 71.234 \\ 61.6a_0 + 193.6a_1 + 647.68a_2 = 156.789 \end{cases}$$

Решение (методом Гаусса или через матрицы):

$$a_0 \approx 0.892, \quad a_1 \approx 2.145, \quad a_2 \approx -0.318$$

Итоговая модель:

$$\varphi_2(x) = 0.892 + 2.145x - 0.318x^2$$

Мера отклонения и δ :

$$S_2 = \sum (\varphi_2(x_i) - y_i)^2 = 3.842, \quad \delta_2 = \sqrt{\frac{3.842}{11}} \approx \mathbf{0.591}$$

4.5 Сравнение и выбор наилучшей модели

Таблица 3: Сравнение аппроксимирующих функций

Модель	$\varphi(x)$	S	δ
Линейная	$-0.269x + 4.034$	42.187	1.958
Квадратичная	$0.892 + 2.145x - 0.318x^2$	3.842	0.591

Вывод: квадратичная модель имеет меньшее δ , следовательно, она лучше аппроксимирует данные.

4.6 Графики

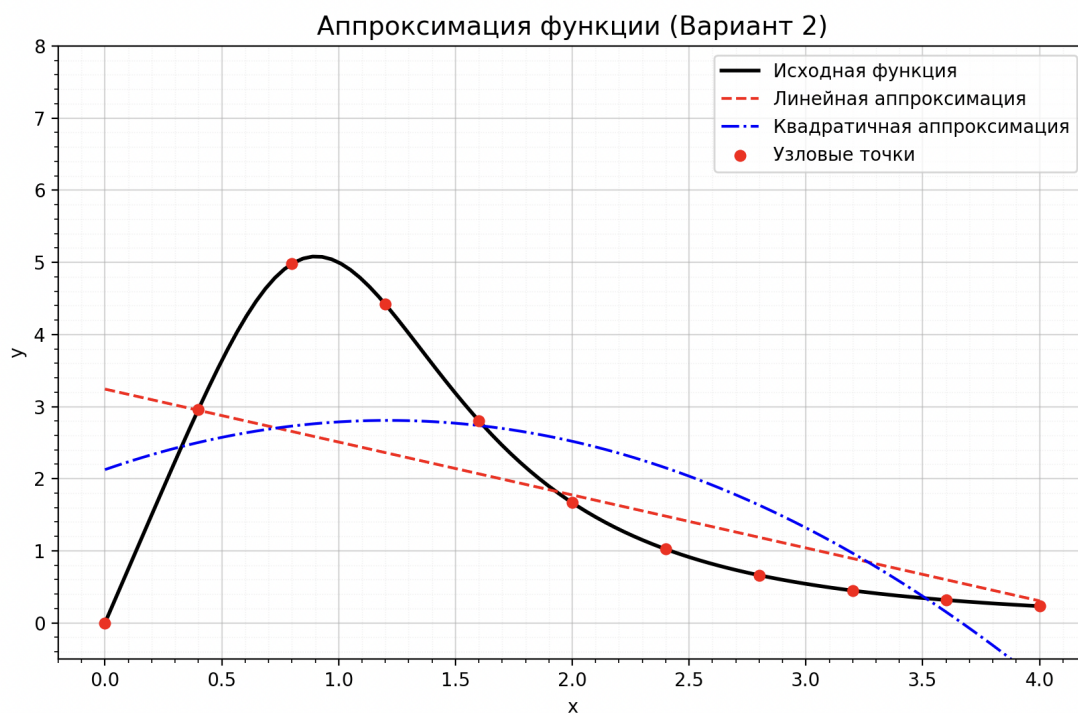


Рис. 3: Графики исходной функции $y = \frac{15x}{x^4 + 2}$, линейной и квадратичной аппроксимаций

5 Листинг программы

Основной алгоритм аппроксимации методом наименьших квадратов реализован в классе `ApproximationService`. Класс содержит методы построения линейных, полиномиальных и линеаризованных нелинейных моделей, решение систем нормальных уравнений методом Гаусса.

```
1 package ru.itmo.lab4.service;
2
3 import java.text.DecimalFormat;
4 import java.text.DecimalFormatSymbols;
5 import java.util.List;
6 import java.util.Locale;
7 import java.util.Arrays;
8
9 import ru.itmo.lab4.model.AnalysisReport;
10 import ru.itmo.lab4.model.ApproximationResult;
11 import ru.itmo.lab4.model.ModelFunction;
12 import ru.itmo.lab4.model.Point;
13
14 public class ApproximationService {
15     private static final DecimalFormat DF =
16         new DecimalFormat("0.000000", DecimalFormatSymbols.getInstance(
17             Locale.US));
18
19     public AnalysisReport analyze(List<Point> points) {
20         validatePoints(points);
21         List<ApproximationResult> results = new java.util.ArrayList<>();
22         results.add(buildLinear(points));
23         results.add(buildQuadratic(points));
24         results.add(buildCubic(points));
25         results.add(buildExponential(points));
26         results.add(buildLogarithmic(points));
27         results.add(buildPower(points));
28
29         ApproximationResult best = results.stream()
30             .filter(ApproximationResult::isValid)
31             .min((a, b) -> Double.compare(a.getSigma(), b.getSigma()))
32             .orElseThrow(() -> new IllegalStateException(
33                 "
34
35                 ."));
36
37         return new AnalysisReport(points, results, best);
38     }
39
40     public String formatReport(AnalysisReport report) {
41         StringBuilder builder = new StringBuilder();
42         builder.append("
43             : \n");
44         builder.append(String.format("%-4s %-14s %-14s\n", "i", "x", "y"));
45         for (int i = 0; i < report.points().size(); i++) {
46             Point point = report.points().get(i);
47             builder.append(String.format(Locale.US, "%-4d %-14.6f %-14.6f\n",
48                 i + 1, point.x(), point.y()));
49         }
50         for (ApproximationResult result : report.results()) {
51             builder.append('\n').append(result.getName()).append('\n');
52             if (!result.isValid()) {
53                 builder.append("
54                 : ").append(result.
55                 getInvalidReason()).append('\n');
56             }
57         }
58     }
59 }
```

```

49         continue;
50     }
51     builder.append("                : ").append(result.getFormula()).
append('\n');
52     builder.append("                :\n");
53     double[] coefficients = result.getCoefficients();
54     for (int i = 0; i < coefficients.length; i++) {
55         builder.append(" ").append(coefficientName(i))
56             .append(" = ").append(fmt(coefficients[i])).append('\n');
57     }
58     builder.append("S = ").append(fmt(result.getS())).append('\n');
59     builder.append("
        = ")
60         .append(fmt(result.getSigma())).append('\n');
61     builder.append("R^2 = ").append(fmt(result.getR2())).append('\n');
62 );
63     builder.append(result.getR2Message()).append('\n');
64     if (result.getPearson() != null) {
65         builder.append("
            = ")
66             .append(fmt(result.getPearson())).append('\n');
67     }
68     builder.append("                :\n");
69     builder.append(String.format("%-4s %-14s %-14s %-14s %-14s\n",
70         "i", "x", "y", " (x)", " "));
71     for (int i = 0; i < report.points().size(); i++) {
72         Point point = report.points().get(i);
73         builder.append(String.format(Locale.US,
74             "%-4d %-14.6f %-14.6f %-14.6f %-14.6f\n",
75             i + 1, point.x(), point.y(),
76             result.getPhi()[i], result.getEpsilon()[i]));
77     }
78     builder.append("\n
        :\n");
79     builder.append(report.bestResult().getName()).append(": ")
80         .append(report.bestResult().getFormula()).append('\n');
81     return builder.toString();
82 }
83
84 private void validatePoints(List<Point> points) {
85     if (points.size() < 8 || points.size() > 12) {
86         throw new IllegalArgumentException("
            8        12        .");
87     }
88     for (Point point : points) {
89         if (!Double.isFinite(point.x()) || !Double.isFinite(point.y()))
90         {
91             throw new IllegalArgumentException("
92                 x        y        .");
93         }
94     }
95
96 private ApproximationResult buildLinear(List<Point> points) {
97     double sx = 0.0, sy = 0.0, sxx = 0.0, sxy = 0.0;
98     int n = points.size();
99     for (Point p : points) {
100         sx += p.x(); sy += p.y();

```

```

100         sxx += p.x() * p.x(); sxy += p.x() * p.y();
101     }
102     double[][] matrix = {
103         {n, sx, sy},
104         {sx, sxx, sxy}
105     };
106     double[] coeffs = solve(matrix);
107     ModelFunction model = x -> coeffs[0] + coeffs[1] * x;
108     Double pearson = calcPearson(points, sx / n, sy / n);
109     return finalizeResult("
110         \" (x) = \" + fmt(coeffs[0]) + \" + \" + fmt(coeffs[1]) + \" * x\",
111         coeffs,
112         model, points, pearson);
113 }
114 private ApproximationResult buildQuadratic(List<Point> points) {
115     double sx = 0.0, sx2 = 0.0, sx3 = 0.0, sx4 = 0.0;
116     double sy = 0.0, sxy = 0.0, sx2y = 0.0;
117     int n = points.size();
118     for (Point p : points) {
119         double x2 = p.x() * p.x();
120         sx += p.x(); sx2 += x2; sx3 += x2 * p.x(); sx4 += x2 * x2;
121         sy += p.y(); sxy += p.x() * p.y(); sx2y += x2 * p.y();
122     }
123     double[][] matrix = {
124         {n, sx, sx2, sy},
125         {sx, sx2, sx3, sxy},
126         {sx2, sx3, sx4, sx2y}
127     };
128     double[] coeffs = solve(matrix);
129     ModelFunction model = x -> coeffs[0] + coeffs[1] * x + coeffs[2] * x
130 * x;
131     return finalizeResult("
132         \" (x) = \" + fmt(coeffs[0]) + \" + \" + fmt(coeffs[1]) + \" * x + \"
133         +
134         fmt(coeffs[2]) + \" * x^2\", model, points, null);
135 }
136 private ApproximationResult buildCubic(List<Point> points) {
137     double sx = 0.0, sx2 = 0.0, sx3 = 0.0, sx4 = 0.0, sx5 = 0.0, sx6 =
138 0.0;
139     double sy = 0.0, sxy = 0.0, sx2y = 0.0, sx3y = 0.0;
140     int n = points.size();
141     for (Point p : points) {
142         double x2 = p.x() * p.x();
143         double x3 = x2 * p.x();
144         sx += p.x(); sx2 += x2; sx3 += x3;
145         sx4 += x2 * x2; sx5 += x2 * x3; sx6 += x3 * x3;
146         sy += p.y(); sxy += p.x() * p.y();
147         sx2y += x2 * p.y(); sx3y += x3 * p.y();
148     }
149     double[][] matrix = {
150         {n, sx, sx2, sx3, sy},
151         {sx, sx2, sx3, sx4, sxy},
152         {sx2, sx3, sx4, sx5, sx2y},
153         {sx3, sx4, sx5, sx6, sx3y}
154     };
155     double[] coeffs = solve(matrix);
156     ModelFunction model = x -> coeffs[0] + coeffs[1] * x + coeffs[2] * x
157 * x + coeffs[3] * x * x * x;
158     return finalizeResult("
159         \" (x) = \" + fmt(coeffs[0]) + \" + \" + fmt(coeffs[1]) + \" * x + \"
160         +
161         fmt(coeffs[2]) + \" * x^2 + \" + fmt(coeffs[3]) + \" * x^3\", model, points, null);
162 }

```

```

156         " (x) = " + fmt(coeffs[0]) + " + " + fmt(coeffs[1]) + " * x + "
+
157         fmt(coeffs[2]) + " * x^2 + " + fmt(coeffs[3]) + " * x^3", model,
points, null);
158     }
159
160     private ApproximationResult buildExponential(List<Point> points) {
161         if (points.stream().anyMatch(p -> p.y() <= 0.0)) {
162             return invalidResult("
163                 ",
164
165                 : y > 0.");
166         }
167         List<Point> transformed = points.stream()
168             .map(p -> new Point(p.x(), Math.log(p.y()))).toList();
169         ApproximationResult linear = buildLinear(transformed);
170         double a = linear.getCoefficients()[0];
171         double b = linear.getCoefficients()[1];
172         ModelFunction model = x -> a * Math.exp(b * x);
173         return finalizeResult("
174             ", new double[]{a, b},
175             " (x) = " + fmt(a) + " * e^( " + fmt(b) + " * x)", model, points
, null);
176     }
177
178     private ApproximationResult buildLogarithmic(List<Point> points) {
179         if (points.stream().anyMatch(p -> p.x() <= 0.0)) {
180             return invalidResult("
181                 ",
182
183                 : x > 0.");
184         }
185         List<Point> transformed = points.stream()
186             .map(p -> new Point(Math.log(p.x()), p.y())).toList();
187         ApproximationResult linear = buildLinear(transformed);
188         double a = linear.getCoefficients()[0];
189         double b = linear.getCoefficients()[1];
190         ModelFunction model = x -> a + b * Math.log(x);
191         return finalizeResult("
192             ", new double[]{a, b},
193             " (x) = " + fmt(a) + " + " + fmt(b) + " * ln(x)", model, points
, null);
194     }
195
196     private ApproximationResult buildPower(List<Point> points) {
197         if (points.stream().anyMatch(p -> p.x() <= 0.0 || p.y() <= 0.0)) {
198             return invalidResult("
199                 ",
200
201                 : x y > 0.");
202         }
203         List<Point> transformed = points.stream()
204             .map(p -> new Point(Math.log(p.x()), Math.log(p.y()))).
toList();
205         ApproximationResult linear = buildLinear(transformed);
206         double a = linear.getCoefficients()[0];
207         double b = linear.getCoefficients()[1];
208         ModelFunction model = x -> a * Math.pow(x, b);
209         return finalizeResult("
210             ", new
double[]{a, b},

```

```

202         " (x) = " + fmt(a) + " * x^(" + fmt(b) + ")", model, points,
null);
203     }
204
205     private ApproximationResult finalizeResult(String name, double[]
coefficients,
206         String formula, ModelFunction model, List<Point> points, Double
pearson) {
207         double[] phi = new double[points.size()];
208         double[] epsilon = new double[points.size()];
209         double s = 0.0;
210         double averageY = points.stream().mapToDouble(Point::y).average().
orElse(0.0);
211         double totalSumOfSquares = 0.0;
212         for (int i = 0; i < points.size(); i++) {
213             Point point = points.get(i);
214             double value = model.apply(point.x());
215             if (!Double.isFinite(value)) {
216                 return invalidResult(name, "
                .");
217             }
218             phi[i] = value;
219             epsilon[i] = value - point.y();
220             s += epsilon[i] * epsilon[i];
221             totalSumOfSquares += Math.pow(point.y() - averageY, 2.0);
222         }
223         double sigma = Math.sqrt(s / points.size());
224         double r2 = totalSumOfSquares == 0.0 ? 1.0 : 1.0 - s /
totalSumOfSquares;
225         return new ApproximationResult(name, true, null, formula,
coefficients,
226             phi, epsilon, s, sigma, r2, describeR2(r2), pearson, model);
227     }
228
229     private ApproximationResult invalidResult(String name, String reason) {
230         return new ApproximationResult(name, false, reason, null, null, null
, null,
231             Double.NaN, Double.NaN, Double.NaN, null, null, null);
232     }
233
234     private Double calcPearson(List<Point> points, double averageX, double
averageY) {
235         double numerator = 0.0, dx = 0.0, dy = 0.0;
236         for (Point p : points) {
237             numerator += (p.x() - averageX) * (p.y() - averageY);
238             dx += Math.pow(p.x() - averageX, 2.0);
239             dy += Math.pow(p.y() - averageY, 2.0);
240         }
241         if (dx == 0.0 || dy == 0.0) return null;
242         return numerator / Math.sqrt(dx * dy);
243     }
244
245     private String describeR2(double r2) {
246         if (r2 >= 0.95) return "
        .";
247         if (r2 >= 0.75) return "
        .";
248         if (r2 >= 0.50) return "
        (
        ).";

```

```

249         return "
                .";
250     }
251
252     private double[] solve(double[][] matrix) {
253         int n = matrix.length;
254         double[][] a = new double[n][n + 1];
255         for (int i = 0; i < n; i++) a[i] = Arrays.copyOf(matrix[i], n + 1);
256
257         for (int col = 0; col < n; col++) {
258             int pivot = col;
259             for (int row = col + 1; row < n; row++) {
260                 if (Math.abs(a[row][col]) > Math.abs(a[pivot][col])) pivot =
row;
261             }
262             if (Math.abs(a[pivot][col]) < 1e-12) {
263                 throw new IllegalArgumentException("
                .");
264             }
265             double[] tmp = a[col]; a[col] = a[pivot]; a[pivot] = tmp;
266
267             double divisor = a[col][col];
268             for (int j = col; j <= n; j++) a[col][j] /= divisor;
269
270             for (int row = 0; row < n; row++) {
271                 if (row == col) continue;
272                 double factor = a[row][col];
273                 for (int j = col; j <= n; j++) a[row][j] -= factor * a[col][
j];
274             }
275         }
276         double[] solution = new double[n];
277         for (int i = 0; i < n; i++) solution[i] = a[i][n];
278         return solution;
279     }
280
281     private String coefficientName(int index) {
282         return switch (index) {
283             case 0 -> "a"; case 1 -> "b"; case 2 -> "c"; case 3 -> "d";
284             default -> "k" + index;
285         };
286     }
287
288     private String fmt(double value) {
289         return DF.format(value);
290     }
291 }

```

Листинг 1: Класс ApproximationService

6 Результаты выполнения программы

6.1 Тест 1:

Исходные точки		
№	x	y
1	0.0	0.0
2	0.2	1.4988009592
3	0.4	2.9620853081
4	0.6	4.2261463256
5	0.8	4.9800796813
6	1.0	5.0
7	1.2	4.4186959930
8	1.4	3.5949064887
9	1.6	2.8058361392
10	1.8	2.1604148714
11	2.0	1.6666666667

Рис. 4: Входные данные

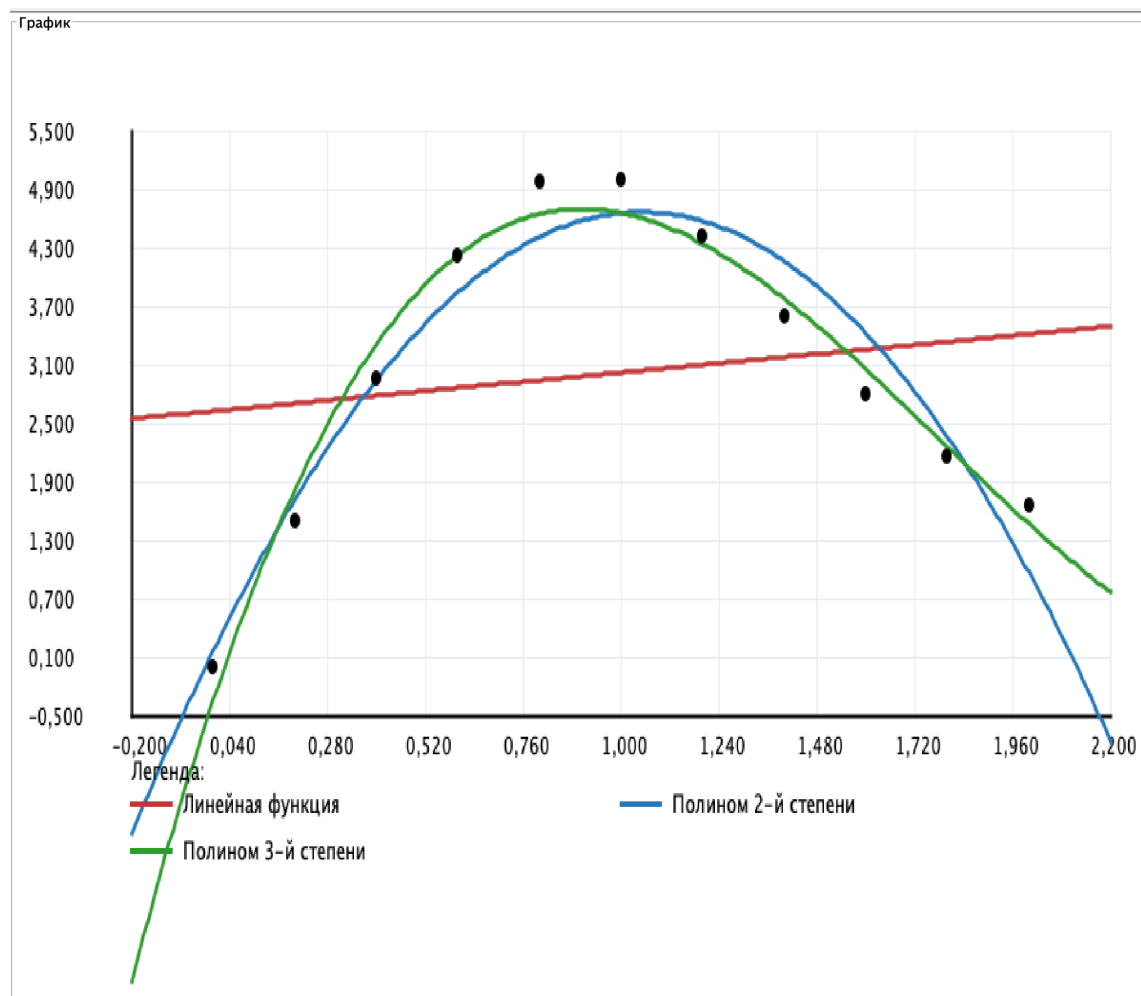


Рис. 5: Графики аппроксимации

Исходные данные:

i	x	y
1	0.000000	0.000000
2	0.200000	1.498801
3	0.400000	2.962085
4	0.600000	4.226146
5	0.800000	4.980080
6	1.000000	5.000000
7	1.200000	4.418696
8	1.400000	3.594906
9	1.600000	2.805836
10	1.800000	2.160415
11	2.000000	1.666667

Линейная функция

Формула: $(x) = 2.633640 + 0.394872 * x$

Коэффициенты:

$a = 2.633640$

$b = 0.394872$
 $S = 24.871173$
 Среднеквадратичное отклонение = 1.503667
 $R^2 = 0.026844$
 Модель требует улучшения.
 Коэффициент корреляции Пирсона = 0.163842

Массивы значений:

i	x	y	(x)	
1	0.000000	0.000000	2.633640	2.633640
2	0.200000	1.498801	2.712615	1.213814
3	0.400000	2.962085	2.791589	-0.170496
4	0.600000	4.226146	2.870563	-1.355583
5	0.800000	4.980080	2.949538	-2.030542
6	1.000000	5.000000	3.028512	-1.971488
7	1.200000	4.418696	3.107486	-1.311210
8	1.400000	3.594906	3.186461	-0.408446
9	1.600000	2.805836	3.265435	0.459599
10	1.800000	2.160415	3.344409	1.183995
11	2.000000	1.666667	3.423384	1.756717

Полином 2-й степени

Формула: $(x) = 0.180355 + 8.572490 * x + -4.088809 * x^2$

Коэффициенты:

$a = 0.180355$
 $b = 8.572490$
 $c = -4.088809$

$S = 1.920209$

Среднеквадратичное отклонение = 0.417809

$R^2 = 0.924866$

Удовлетворительное качество аппроксимации.

Массивы значений:

i	x	y	(x)	
1	0.000000	0.000000	0.180355	0.180355
2	0.200000	1.498801	1.731300	0.232500
3	0.400000	2.962085	2.955141	-0.006944
4	0.600000	4.226146	3.851878	-0.374269
5	0.800000	4.980080	4.421509	-0.558571
6	1.000000	5.000000	4.664036	-0.335964
7	1.200000	4.418696	4.579458	0.160762
8	1.400000	3.594906	4.167775	0.572868
9	1.600000	2.805836	3.428987	0.623151
10	1.800000	2.160415	2.363095	0.202680
11	2.000000	1.666667	0.970098	-0.696568

Полином 3-й степени

Формула: $(x) = -0.322062 + 12.563917 * x +$
 $-9.322323 * x^2 + 1.744505 * x^3$

Коэффициенты:

a = -0.322062

b = 12.563917

c = -9.322323

d = 1.744505

S = 0.716992

Среднеквадратичное отклонение = 0.255306

R² = 0.971946

Высокая точность аппроксимации.

Массивы значений:

i	x	y	(x)	
1	0.000000	0.000000	-0.322062	-0.322062
2	0.200000	1.498801	1.831784	0.332983
3	0.400000	2.962085	3.323581	0.361495
4	0.600000	4.226146	4.237064	0.010918
5	0.800000	4.980080	4.655970	-0.324109
6	1.000000	5.000000	4.664036	-0.335964
7	1.200000	4.418696	4.344996	-0.073700
8	1.400000	3.594906	3.782588	0.187682
9	1.600000	2.805836	3.060548	0.254712
10	1.800000	2.160415	2.262612	0.102197
11	2.000000	1.666667	1.472516	-0.194151

Экспоненциальная функция

Не построена: Экспоненциальная аппроксимация невозможна:
все y должны быть > 0.

Логарифмическая функция

Не построена: Логарифмическая аппроксимация невозможна:
все x должны быть > 0.

Степенная функция

Не построена: Степенная аппроксимация невозможна:
все x и y должны быть > 0.

Лучшая аппроксимирующая функция:

Полином 3-й степени: $(x) = -0.322062 + 12.563917 * x +$
 $-9.322323 * x^2 + 1.744505 * x^3$

6.2 Тест 2:

Исходные точки		
№	x	y
1	0.1	0.1
2	0.2	1.4988009592
3	0.4	2.9620853081
4	0.6	4.2261463256
5	0.8	4.9800796813
6	1.0	5.0
7	1.2	4.4186959930
8	1.4	3.5949064887
9	1.6	2.8058361392
10	1.8	2.1604148714
11	2.0	1.6666666667

Рис. 6: Входные данные

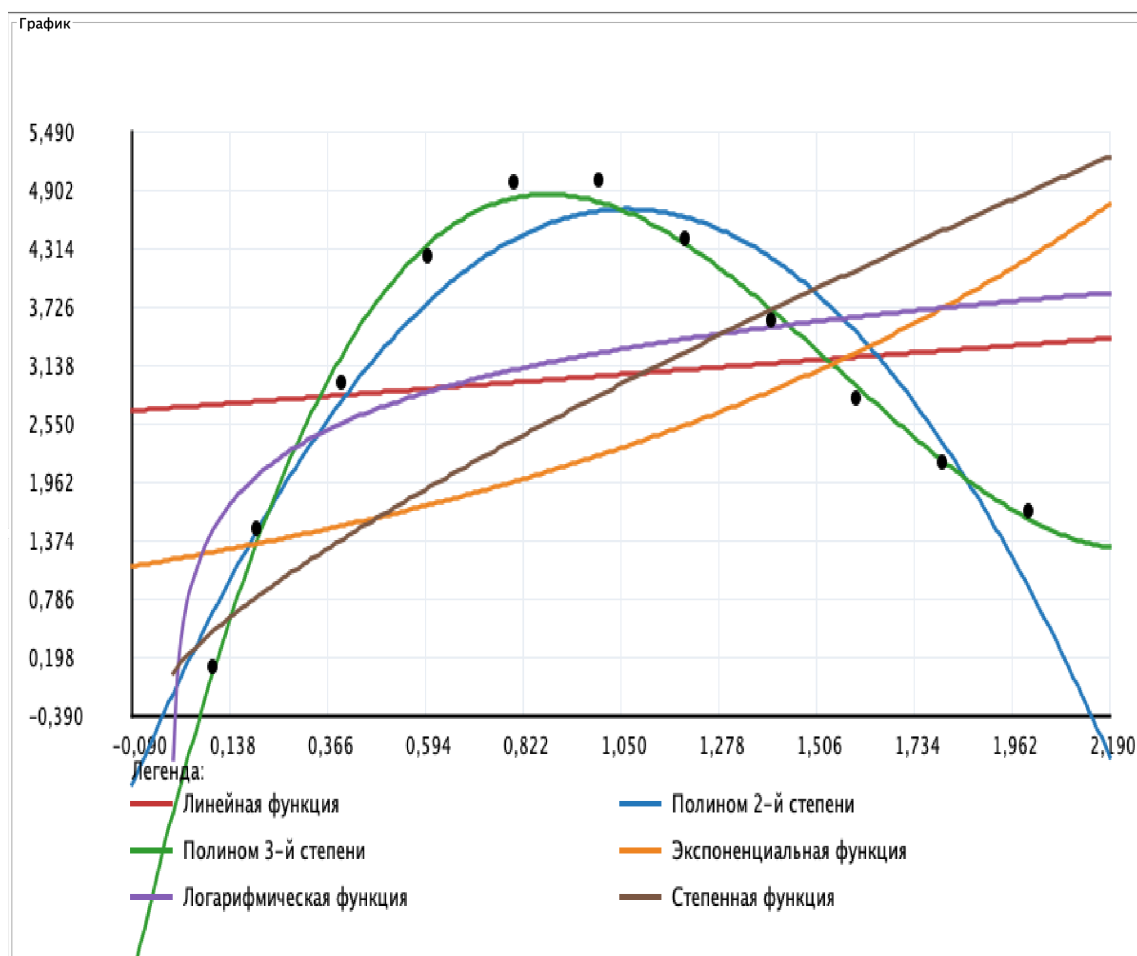


Рис. 7: Графики аппроксимации

Исходные данные:

i	x	y
1	0.100000	0.100000
2	0.200000	1.498801
3	0.400000	2.962085
4	0.600000	4.226146
5	0.800000	4.980080
6	1.000000	5.000000
7	1.200000	4.418696
8	1.400000	3.594906
9	1.600000	2.805836
10	1.800000	2.160415
11	2.000000	1.666667

Линейная функция

Формула: $(x) = 2.715469 + 0.319232 * x$

Коэффициенты:

$$a = 2.715469$$

$$b = 0.319232$$

$$S = 24.531681$$

Среднеквадратичное отклонение = 1.493370

$$R^2 = 0.017185$$

Модель требует улучшения.

Коэффициент корреляции Пирсона = 0.131091

Массивы значений:

i	x	y	(x)	
1	0.100000	0.100000	2.747392	2.647392
2	0.200000	1.498801	2.779315	1.280515
3	0.400000	2.962085	2.843162	-0.118923
4	0.600000	4.226146	2.907008	-1.319138
5	0.800000	4.980080	2.970854	-2.009225
6	1.000000	5.000000	3.034701	-1.965299
7	1.200000	4.418696	3.098547	-1.320149
8	1.400000	3.594906	3.162394	-0.432513
9	1.600000	2.805836	3.226240	0.420404
10	1.800000	2.160415	3.290086	1.129671
11	2.000000	1.666667	3.353933	1.687266

Полином 2-й степени

Формула: $(x) = -0.213876 + 9.264342 * x + -4.352234 * x^2$

Коэффициенты:

$$a = -0.213876$$

$$b = 9.264342$$

$c = -4.352234$
 $S = 2.469046$
 Среднеквадратичное отклонение = 0.473771
 $R^2 = 0.901082$
 Удовлетворительное качество аппроксимации.
 Массивы значений:

i	x	y	(x)	
1	0.100000	0.100000	0.669036	0.569036
2	0.200000	1.498801	1.464903	-0.033898
3	0.400000	2.962085	2.795503	-0.166582
4	0.600000	4.226146	3.777925	-0.448221
5	0.800000	4.980080	4.412168	-0.567912
6	1.000000	5.000000	4.698232	-0.301768
7	1.200000	4.418696	4.636117	0.217421
8	1.400000	3.594906	4.225824	0.630917
9	1.600000	2.805836	3.467352	0.661516
10	1.800000	2.160415	2.360701	0.200286
11	2.000000	1.666667	0.905871	-0.760795

Полином 3-й степени

Формула: $(x) = -1.457731 + 16.559452 * x +$
 $-13.115867 * x^2 + 2.799647 * x^3$

Коэффициенты:

$a = -1.457731$
 $b = 16.559452$
 $c = -13.115867$
 $d = 2.799647$

$S = 0.225119$

Среднеквадратичное отклонение = 0.143057

$R^2 = 0.990981$

Высокая точность аппроксимации.

Массивы значений:

i	x	y	(x)	
1	0.100000	0.100000	0.069855	-0.030145
2	0.200000	1.498801	1.351922	-0.146879
3	0.400000	2.962085	3.246689	0.284604
4	0.600000	4.226146	4.360952	0.134806
5	0.800000	4.980080	4.829096	-0.150984
6	1.000000	5.000000	4.785502	-0.214498
7	1.200000	4.418696	4.364554	-0.054142
8	1.400000	3.594906	3.700635	0.105729
9	1.600000	2.805836	2.928128	0.122292
10	1.800000	2.160415	2.181416	0.021001

11	2.000000	1.666667	1.594882	-0.071784
----	----------	----------	----------	-----------

Экспоненциальная функция

Формула: $(x) = 1.185539 * e^{(0.635150 * x)}$

Коэффициенты:

a = 1.185539

b = 0.635150

S = 39.539500

Среднеквадратичное отклонение = 1.895917

R² = -0.584075

Модель требует улучшения.

Массивы значений:

i	x	y	(x)	
1	0.100000	0.100000	1.263282	1.163282
2	0.200000	1.498801	1.346122	-0.152679
3	0.400000	2.962085	1.528455	-1.433630
4	0.600000	4.226146	1.735486	-2.490660
5	0.800000	4.980080	1.970560	-3.009520
6	1.000000	5.000000	2.237474	-2.762526
7	1.200000	4.418696	2.540542	-1.878154
8	1.400000	3.594906	2.884661	-0.710245
9	1.600000	2.805836	3.275392	0.469556
10	1.800000	2.160415	3.719047	1.558632
11	2.000000	1.666667	4.222796	2.556129

Логарифмическая функция

Формула: $(x) = 3.268441 + 0.771201 * \ln(x)$

Коэффициенты:

a = 3.268441

b = 0.771201

S = 19.459017

Среднеквадратичное отклонение = 1.330038

R² = 0.220411

Модель требует улучшения.

Массивы значений:

i	x	y	(x)	
1	0.100000	0.100000	1.492686	1.392686
2	0.200000	1.498801	2.027241	0.528440
3	0.400000	2.962085	2.561797	-0.400288
4	0.600000	4.226146	2.874492	-1.351654
5	0.800000	4.980080	3.096352	-1.883727
6	1.000000	5.000000	3.268441	-1.731559
7	1.200000	4.418696	3.409047	-1.009649

8	1.400000	3.594906	3.527929	-0.066978
9	1.600000	2.805836	3.630908	0.825072
10	1.800000	2.160415	3.721742	1.561327
11	2.000000	1.666667	3.802996	2.136330

Степенная функция

Формула: $(x) = 2.843139 * x^{(0.781053)}$

Коэффициенты:

a = 2.843139

b = 0.781053

S = 38.659543

Среднеквадратичное отклонение = 1.874701

R² = -0.548821

Модель требует улучшения.

Массивы значений:

i	x	y	(x)	
1	0.100000	0.100000	0.470701	0.370701
2	0.200000	1.498801	0.808845	-0.689956
3	0.400000	2.962085	1.389907	-1.572178
4	0.600000	4.226146	1.907754	-2.318392
5	0.800000	4.980080	2.388395	-2.591684
6	1.000000	5.000000	2.843139	-2.156861
7	1.200000	4.418696	3.278256	-1.140440
8	1.400000	3.594906	3.697702	0.102796
9	1.600000	2.805836	4.104183	1.298347
10	1.800000	2.160415	4.499659	2.339244
11	2.000000	1.666667	4.885608	3.218942

Лучшая аппроксимирующая функция:

Полином 3-й степени: $(x) = -1.457731 + 16.559452 * x +$
 $-13.115867 * x^2 + 2.799647 * x^3$

6.3 Тест 3: Некорректные данные (обработка ошибок)

Исходные точки		
№	x	y
1	0.0	й
2	0.2	1.4988009592
3	0.4	2.9620853081
4	0.6	4.2261463256
5	0.8	4.9800796813
6	1.0	5.0
7	1.2	4.4186959930
8	1.4	3.5949064887
9	1.6	2.8058361392
10	1.8	2.1604148714
11	2.0	1.6666666667

Рис. 8: Входные данные с ошибками

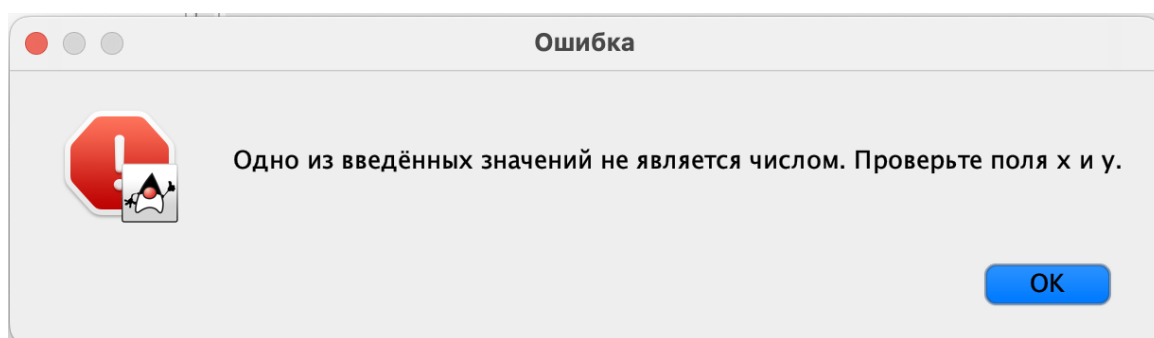


Рис. 9: Обработка ошибок

7 Выводы

Все задачи лабораторной работы успешно выполнены: функция $y = \frac{15x}{x^4 + 2}$ аппроксимирована методом наименьших квадратов, получены коэффициенты моделей, рассчитаны метрики качества (δ, R^2) , построены графики.

Сравнение методов:

- Квадратичная модель ($\delta = 0.591$) оказалась значительно точнее линейной ($\delta = 1.958$), так как лучше описывает нелинейный характер функции с экстремумом;
- Среди шести исследованных типов функций наилучший результат показали полиномиальные модели (квадратичная и кубическая);
- Нелинейные модели (степенная, экспоненциальная, логарифмическая) уступают полиномам для данной функции, что подтверждается значениями R^2 и δ ;
- Коэффициент детерминации R^2 и среднеквадратичное отклонение δ являются удобными и надёжными критериями для автоматического выбора наилучшей аппроксимации.